交付物 1：架构候选方案对比表

1. 评估背景与核心约束

针对 CampusHub 校园 Web 应用程序的研发需求，团队在进行架构选型时，严格基于以下核心约束条件进行了综合考量：

团队规模与能力：4 名大二软件工程专业学生，具备基础的 Web 开发能力（Vue 3 + Spring Boot），但缺乏高并发与复杂分布式系统的实战经验。

开发周期：严格限制在 10 周内完成核心 MVP（最小可行性产品）的交付。

质量属性优先级：首要目标为**可维护性**（便于后续交接与功能迭代），其次为**开发效率**（确保按时交付），系统需具备一定的**可扩展性**以应对校园突发流量。

2. 候选架构方案多维度对比

基于上述约束，团队对三种主流架构方案进行了深入调研与对比分析，具体评估如下：

对比维度	方案 A：传统单体分层架构 (Monolithic MVC)	方案 B：模块化单体架构 (Modular Monolith)	方案 C：微服务架构 (Microservices)
开发复杂度	**最低**。 代码集中于单一工程，采用经典 `Controller-Service-Repository` 模式，本地开发仅需启动单个应用实例和数据库即可进行联调，无需处理复杂的网络通信。	**较低**。 在单体架构的基础上，内部实行严格的“按功能分包 (Package-by-Feature)”。需要前期投入一定时间进行领域边界划分，但整体开发心智负担依然较小。	**极高**。 需要处理分布式事务、服务间 RPC/HTTP 通信、API 网关路由、分布式追踪及跨域认证等一系列分布式系统固有的基础设施难题。
可维护性	**中等**。 项目初期维护成本低（单点日志、单点部署）。但随着业务增长，极易演变为代码高度耦合的“面条式”代码，后期交接与重构困难。	**极高**。 物理上保持单体部署的简易性，逻辑上实现了高度的业务隔离。严禁跨模块直接访问数据库层，有效防止代码劣化，极其利于后续团队接	**较低**。 排查单一业务请求报错可能需要跨越多个服务节点，依赖完善的链路追踪系统。对于缺乏 DevOps 经验的学生团队而言，运维与排障成本呈指数级上

		手与长期维护。	升。
可扩展性	**中等**。 主要依赖增加服务器实例进行整体的水平扩展（集群部署 + 负载均衡），无法针对单一高频业务（如活动抢票）进行独立扩容。	**良好**。 逻辑边界清晰，模块间通过接口契约通信。未来若项目规模扩大，可将特定模块（如高并发的活动模块）平滑剥离为独立的微服务。	**极高**。 支持针对特定高并发服务单独进行弹性扩容，且不影响其他业务模块的稳定性。具备最佳的长期可扩展能力。
系统性能	**极高**。 模块间的通信为内存级的本地方法调用，零网络传输开销，单次请求的绝对延迟极低。	**极高**。 同为内存级方法调用，响应速度极快，足以应对百万级用户量的常规校园应用场景。	**较低**。 单一前端请求可能需在后端跨越多个微服务，经过多次网络序列化与反序列化，系统整体延迟显著增加。
团队学习成本	**最低**。 团队成员只需掌握基础的 Spring Boot 注解、MyBatis/JPA 和 MVC 思想即可快速上手，契合现有技术储备。	**较低**。 在掌握基础框架的前提下，仅需额外学习模块化设计规范、包隔离原则及接口契约设计，学习曲线平缓。	**极高**。 需额外学习 Spring Cloud 全家桶、消息队列、服务注册与发现、容器编排（K8s）等，极易导致前期陷入“技术踩坑”期。
10 周内交付可行性	**极高**。 架构简单，能迅速进入业务逻辑编写阶段，确保 10 周内按时结项。	**极高**。 兼顾了快速交付与代码整洁，能在保证进度的同时提升代码质量，是最平衡的选择。	**极低（几乎为零）**。 极大概率在开发中后期仍受困于服务间通信与部署环境问题，导致最终只能交付半成品，面临极大的延期风险。

3. 对比总结

通过上述多维度对比，**微服务架构（方案 C）虽然在可扩展性上表现优异，但其极高的开发复杂度与学习成本严重违背了“10 周交付”及“团队经验受限”的核心约束，故被坚决排除**。

在**传统单体分层（方案 A）与模块化单体（方案 B）之间，方案 B 在仅增加微小设计成本的前提下，大幅提升了系统的可维护性**与**未来演进能力**，完美契合本项目“可维护性优先”的质量目标。因此，方案 B 成为本项目的